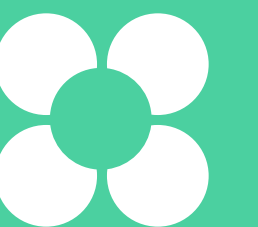
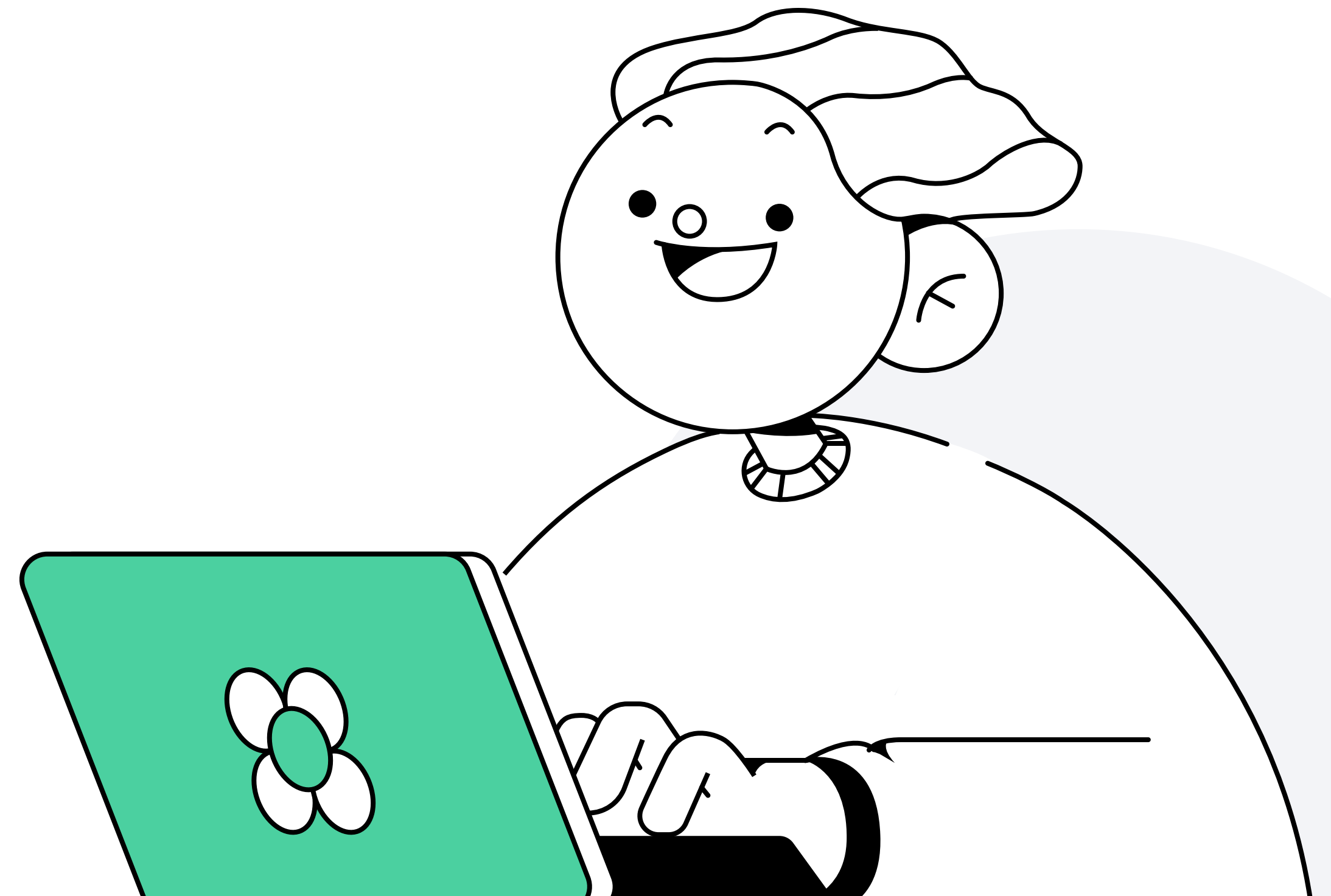


Коллекции HashSet и HashMap

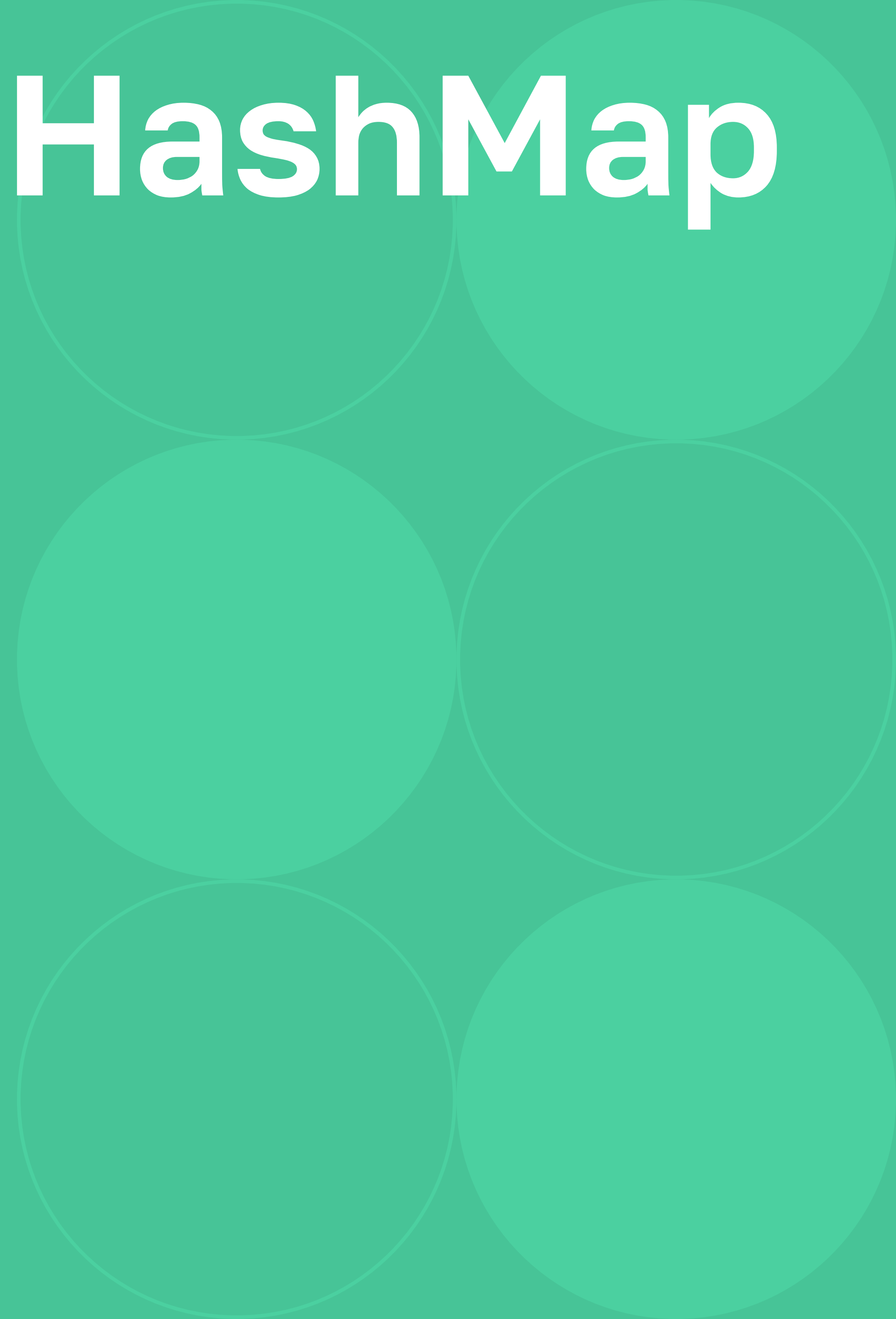


План занятия

- ① Хеширование и HashMap
- ② Множества и HashSet



Хеширование и HashMap



Задача

Дано: мы пишем метод для получения списка водительских удостоверений, выданных в любом конкретном городе. Каждая строка прочитанных данных о водительском удостоверении содержит следующий набор полей:

- идентификатор водительского удостоверения
- идентификатор водителя
- дата выдачи удостоверения
- срок действия
- набор категорий

Вопрос

В какой структуре данных можно хранить такие данные при условии, что эти данные являются объектами, полученными из базы данных или прочитанными из файла, и нам требуется находить каждого водителя по его номеру удостоверения?



Ответ

Можно хранить объекты в массивах или списках (в зависимости от метода дальнейшего использования этих данных). Такой способ возможен, но для поиска нужного элемента в таких коллекциях, например, по номеру водительского удостоверения, нужно будет перебрать все элементы этой коллекции. Это будет работать медленно уже на коллекциях размером в несколько тысяч элементов. Сложность такого поиска будет равна $O(n)$.

Для ускорения поиска объектов по идентификатору были разработаны коллекции на основе **хеш-таблиц**. Такие таблицы позволяют ускорить поиск элементов в них до сложности $O(1)$

Хеш-таблица

Хеш-таблица — это структура данных, представляющая собой ассоциативный массив, то есть структура, хранящая пары «ключ-значение».

В хеш-таблице можно хранить пары вида **«идентификатор — объект»**, **«название — объект»**, **«номер — объект»** и т. д.

Поддерживает стандартные операции:

- добавление пары
- поиск объекта по ключу
- удаление по ключу

Любая хеш-таблица, как следует из её названия, основана на хешировании

Что такое хеширование

Хеширование — это превращение входных данных произвольной длины в выходную строку фиксированной длины или число.

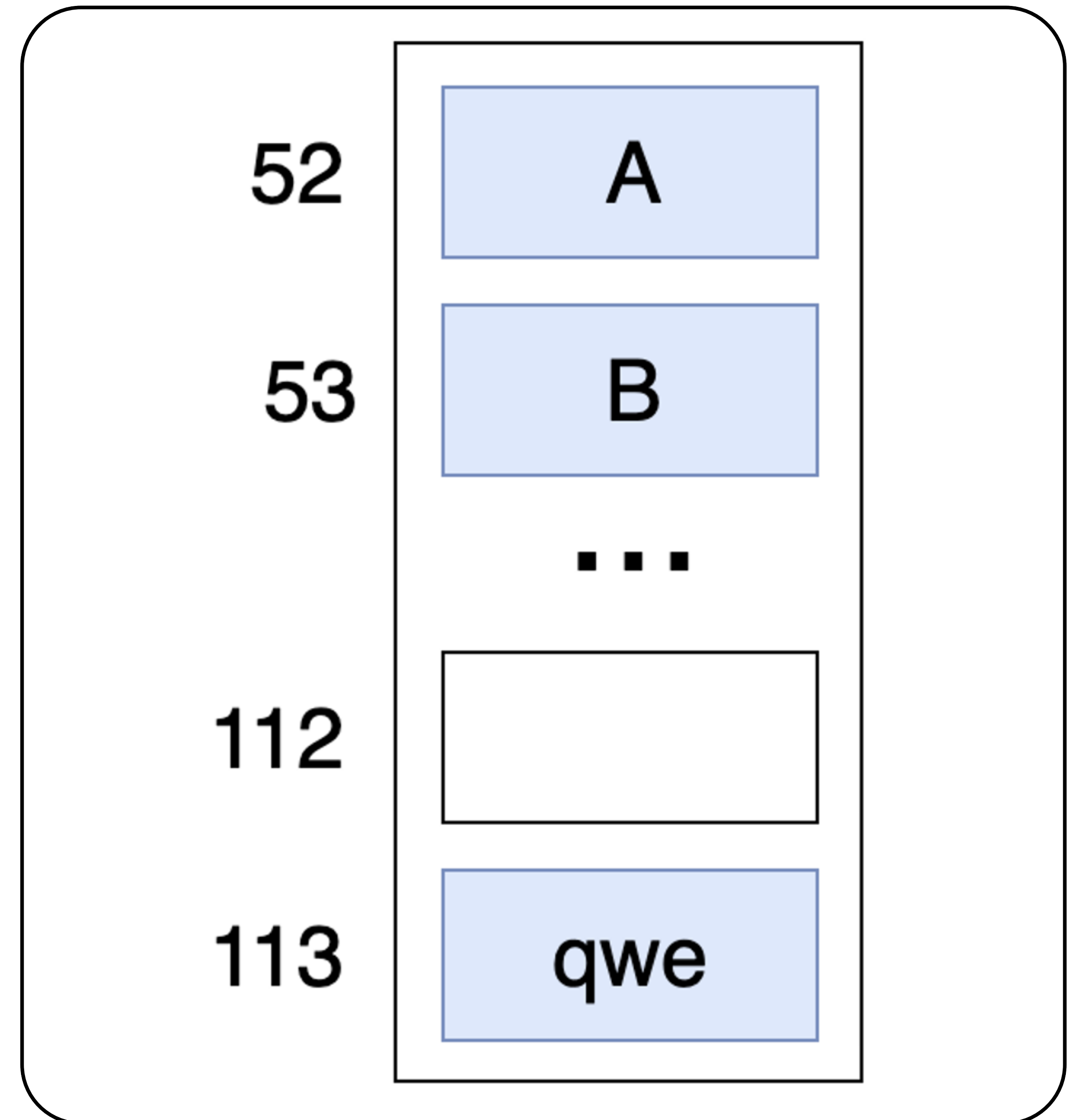
Пример:

```
"A".hashCode()    // 65  
"B".hashCode()    // 66  
"qwe".hashCode()  // 112383
```

Что такое хеширование

Хеш-таблица использует этот код, чтобы найти элемент без перебора всех ключей.

В простейшем случае по хеш-коду определяется индекс в массиве (номер ячейки), по которому находится элемент



Хеширование

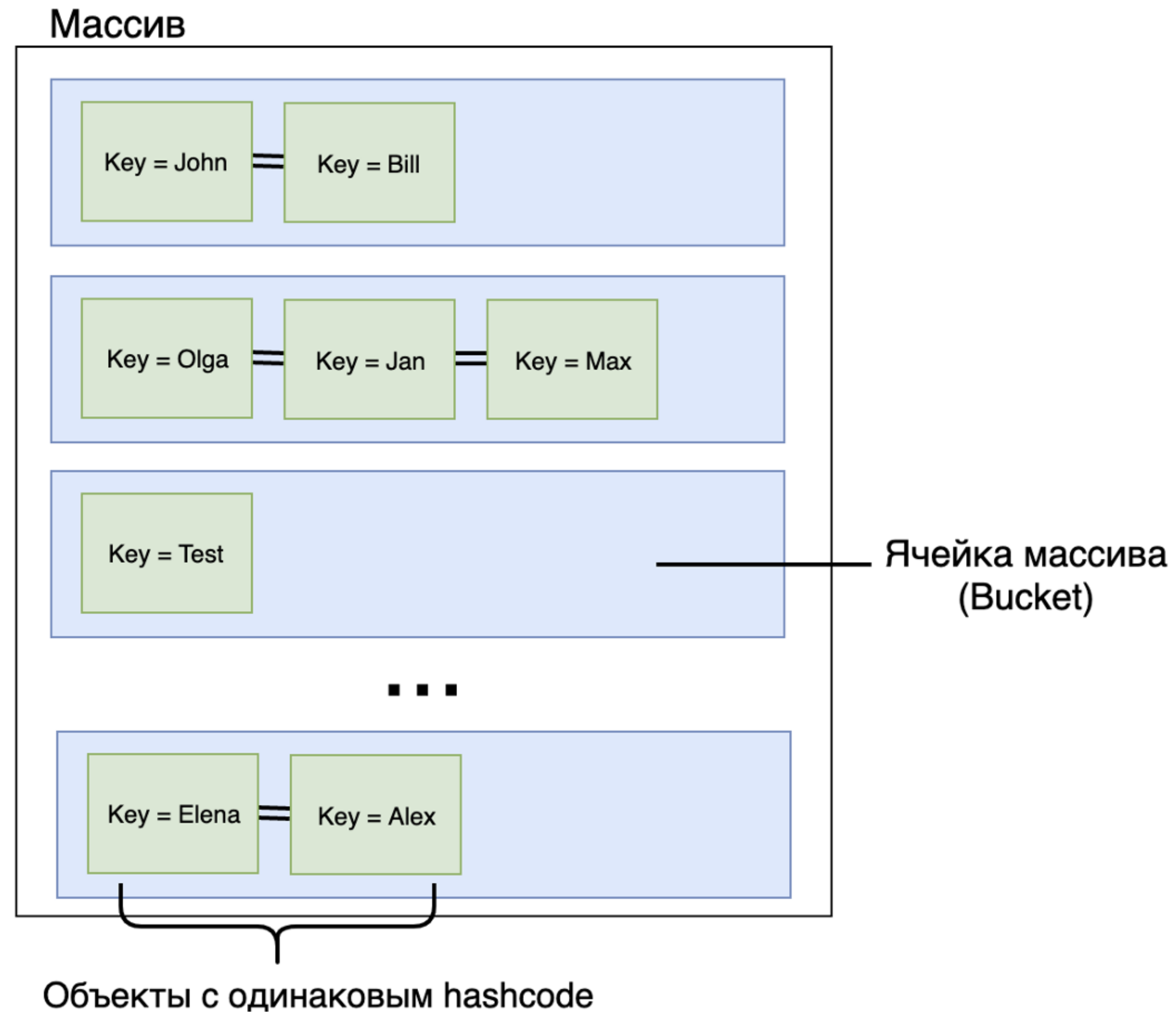
Для одинаковых строк или любых других объектов вычисляется один и тот же хеш-код.

Но также возможна ситуация, когда для разных строк или любых других объектов вычисляется один и тот же хеш-код. Такое явление называется **коллизией**.

Для разрешения коллизий используются разные методы.

В Java, **если по одному индексу, определённом по хешкоду, находятся несколько элементов, то нужный находится с помощью вызовов equals** на каждом из элементов, находящихся в этой ячейке массива

Как выделить инвестирование



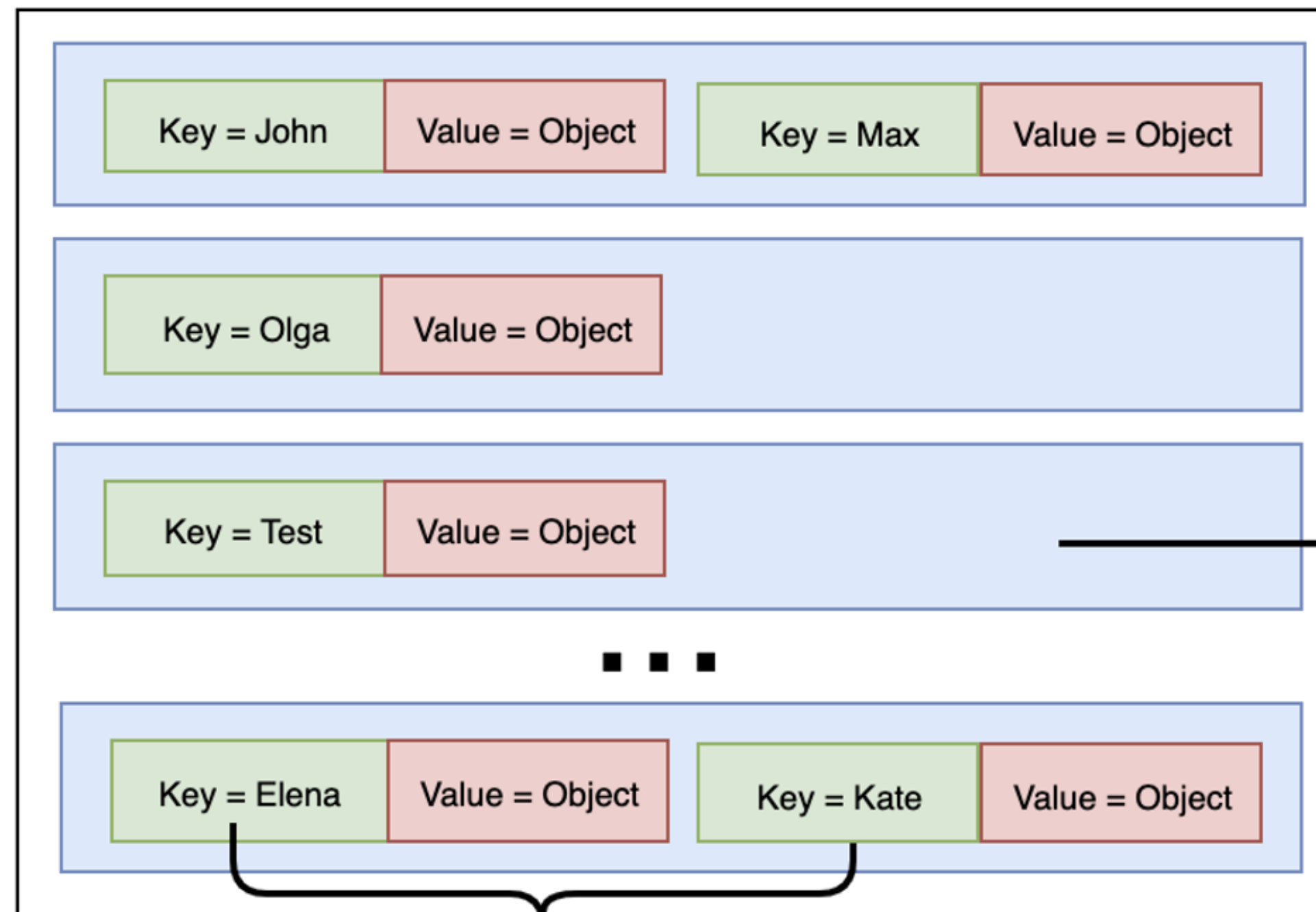
Коллекция HashMap

Чтобы хранить объект с условием частого поиска, изменения, удаления, добавления элементов в этой коллекции по идентификатору этого объекта, в языке Java рекомендуется использовать коллекцию **HashMap**, так как именно эта коллекция основана на хеш-таблицах в языке Java.

Визуально коллекцию **HashMap** можно представить как массив, в котором в качестве ключа используется идентификатор, а в качестве значения — объект, связанный с этим идентификатором

Коллекция HashMap

Массив



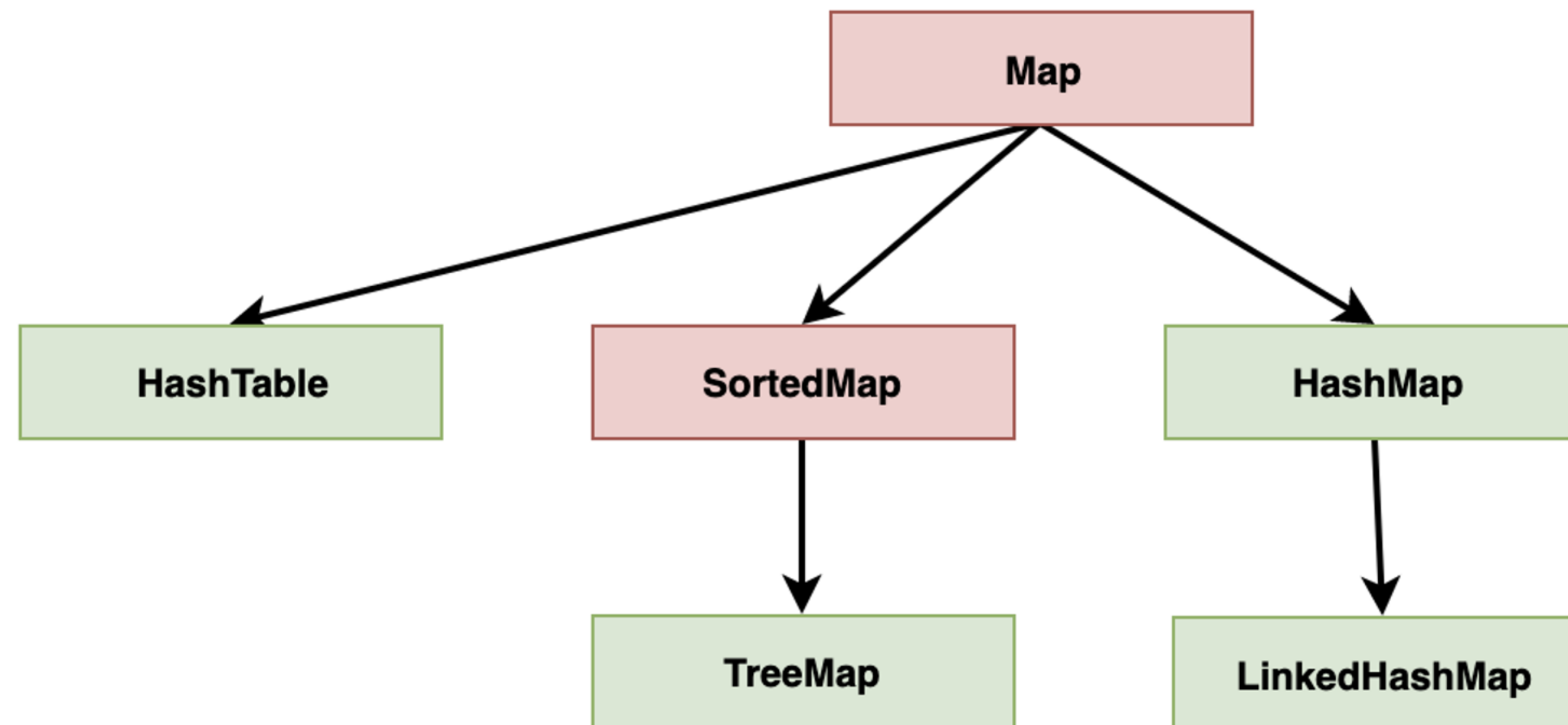
Ячейка
массива
(Bucket)

Объекты с одинаковым hashCode

Номер ячейки массива, в котором будет храниться пара «ключ-значение», вычисляется на основе хеш-кода ключа

Интерфейс Map

Коллекция **HashMap** реализует базовый интерфейс **Map<K,V>**, который можно представить как словарь или справочник, содержащий сопоставление ключей с соответствующими им значениями:



Ключевые методы интерфейса Map

- **put(k,v)** — добавить элемент *v* с ключом *k*
- **putAll(otherMap)** — добавить все элементы из другой Map
- **get(k)** — вернёт значение по ключу *k*; *null*, если такого ключа нет
- **remove(k)** — удаляет элемент с заданным ключом, напоследок возвращает его значение
- **containsKey(k)** — вернёт *true*, если такой ключ используется в map
- **containsValue(v)** — вернёт *true*, если значение хоть раз встречается в map. Внимание: $O(n)$

Ключевые методы интерфейса Map

- **size()** — количество элементов в map
- **isEmpty()** — `size() == 0`
- **clear()** — удалить всё
- **keySet()** — возвращает Set, состоящий из ключей данной коллекции
- **values()** — возвращает Collection, состоящий из значений данной коллекции
- **entrySet()** — возвращает Set, состоящий из пар **<Ключ, Значение> (Entry)**

Особенности реализации HashMap

Чтобы скорость выполнения операций вставки, удаления и поиска при работе с **HashMap** на произвольных классах языка Java была максимально приближена к $O(1)$, необходимо переопределить методы **equals()** и **hashCode()**, соблюдая нижеописанный контракт, и контролировать соблюдение следующих свойств:

- 1 вызов метода **hashCode** на одном и том же объекте должен возвращать одно и то же значение
- 2 если два объекта одинаковые (**equals=true**), то у них должен быть одинаковый хеш-код
- 3 если объекты не одинаковые, то не обязательно, чтобы хеш-код был разным.
Но разный хеш для разных объектов значительно улучшает производительность при использовании хеш-таблиц

Особенности реализации HashMap

Рассмотрим на примере.

Создадим новый тип **UserInfo** и реализуем **HashCode**:

```
import java.util.Objects;

public class UserInfo {
    protected String name;
    protected String surname;
    protected int age;

    public UserInfo(String name, int age) {
        this.name = name;
        this.age = age;
    }
    // реализуем HashCode с помощью метода Objects.hash
    @Override
    public int hashCode() {
        return Objects.hash(name, surname);
    }
}
```


Особенности реализации HashMap

```
// переопределим метод UserInfo  
  
@Override  
public boolean equals(Object obj) {  
    UserInfo o = (UserInfo) obj;  
    return name.equals(o.name) && surname.equals(o.surname);  
}  
}
```

Особенности реализации HashMap

Если изменить ключ (объекта), добавленный в коллекцию **HashMap**, например, изменить поле, которое участвует в вычислении **hashCode**, то в **HashMap** могут оказаться два одинаковых объекта, что способно привести к ошибкам, которые трудно обнаружить.

Для решения такой проблемы в **HashMap** рекомендуется использовать **неизменяемые (immutable) объекты**. Неизменяемым называется класс поля объектов, который нельзя изменить после создания.

Для создания неизменяемого объекта нужно, чтобы поля класса, из которого создаётся объект, были помечены модификатором **final**

Пример переопределения метода equals

```
public boolean equals(Object obj) {  
    // Сравним с собой  
    if (obj == this) {  
        return true;  
    }  
    // Проверим тип  
    if (obj == null || obj.getClass() != this.getClass()) {  
        return false;  
    }  
    // Приведём тип  
    Type other = (Type) obj;  
    // Сравним значения поля (не забывая проверку на null)  
    return id == other.id  
        && (this.field1 == other.field1 || (this.field1 != null && this.field1.equals(other.getField1())))  
        && ... ;  
}
```

Пример переопределения метода hashCode

Вручную:

```
@Override
public int hashCode() {
    int result = 17; //Простое число
    result = 31 * result + field1.hashCode();
    result = 31 * result + intField;
    result = 31 * result + field2.hashCode();
    return result;
}
```

Используя стандартную библиотеку:

```
@Override
public int hashCode() {
    return Objects.hash(field1, field2, intField);
}
```

Пример использования HashMap

Создадим коллекцию **HashMap**, где в качестве ключа используем тип строки (имя), а в качестве значения — тип **integer** (возраст). Попросим Java перебрать все пары «ключ-значение» в map:

```
Map<String, Integer> map = new HashMap<>();
    map.put("Petya", 8);
    map.put("Tanya", 15);

    for (String key : map.keySet()) {
        int value = map.get(key);
        System.out.println(key + " = " + value);
    }

    for (Map.Entry<String, Integer> kv : map.entrySet()) {
        System.out.println(kv.getKey() + " = " + kv.getValue());
    } // Petya = 8 Tanya = 15
```


Пример использования HashMap

null также можно использовать в качестве ключа.

Пример добавления и извлечения объекта с ключом, равным **null**.

```
Map<String, Integer> ages = new HashMap<>();
    map.put("Petya", 8);
    map.put("Tanya", 15);
ages.put(null, 125);
// Добавили значение с ключом null

System.out.println(ages);
// {null=125, Petya=8, Tanya=15}
```

Ещё одна задача

Дано: напомним метод для проверки наличия у водителя нужной категории вождения (A, B, C, D, E).

Вопрос: какую коллекцию стоит в этом случае использовать для хранения категорий, чтобы можно было быстро искать наличие категории у водителя?



Ответ

Как и в предыдущей задаче, возможными вариантами решения могли бы быть массивы или списки. Но проблема медленного поиска (полный перебор) по-прежнему актуальна.

Можно попробовать ввести **HashMap**, но тогда у каждой категории должен быть обязательно добавлен идентификатор. Однако идентификатора может не быть или объект может быть небольшим. Если мы будем использовать в качестве ключа все поля объекта в коллекции, по факту мы будем использовать сам объект как ключ.

Например, если мы храним строки в коллекции, то и ключ, и значение будут строками. Избыточно хранить две строки для одного небольшого объекта. Если объект небольшой, то проще хранить только одну строку. В этом случае используют **множества**. Это ещё один вид коллекции для хранения

Множества и HashSet



Множество

В математике: набор или совокупность объектов, которые обладают общим для них свойством. Над математическим множеством можно выполнять операции: объединение, пересечение, разность и т. д.

В программировании: структура данных, которая хранит объекты определённого типа. При этом порядок объектов не поддерживается. Обычно можно выполнять стандартные математические операции над множествами.

В Java структура данных множества определяет коллекцию (массив) неповторяющихся элементов: **{ a, b, c, d, e }** — множество символов.

Одна из наиболее часто используемых коллекций, описывающих множества, — коллекция **HashSet**, которая в свою очередь основана на **HashMap** и, как следствие, основана на хеш-таблицах

Внутренняя реализация HashSet

HashSet делегирует все свои методы объекту **HashMap**, созданному внутри своей реализации. В качестве ключа для этой коллекции используется сам объект, а в качестве значения — объект-заглушка

Код из JDK

ooo

```
public class HashSet<E>
    extends AbstractSet<E>
    implements Set<E>, Cloneable, java.io.Serializable
{
    static final long serialVersionUID = -5024744406713321676L;

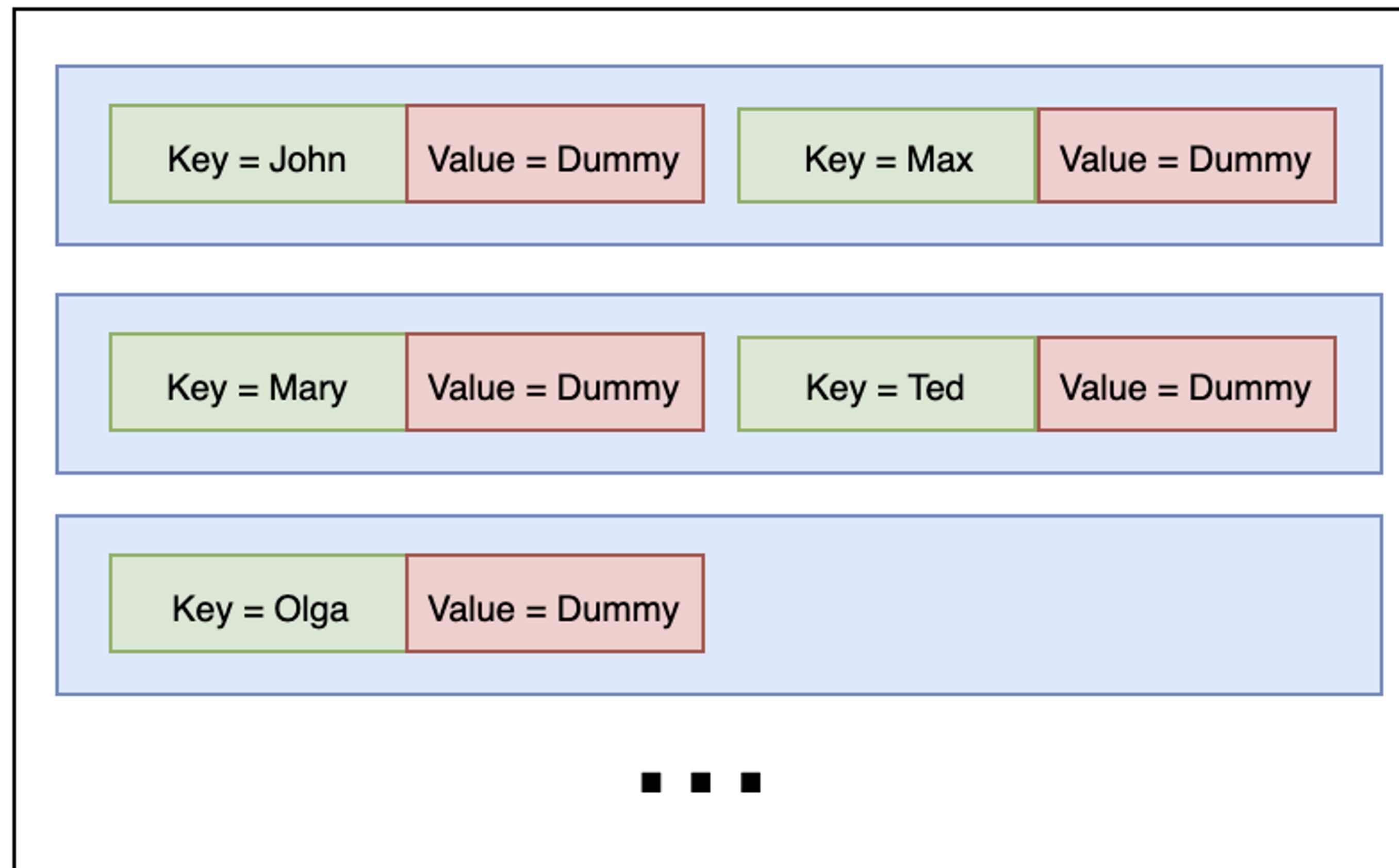
    private transient HashMap<E, Object> map;

    // Dummy value to associate with an Object in the backing Map
    private static final Object PRESENT = new Object();

    /**
     * Constructs a new, empty set; the backing {@code HashMap} instance has
     * default initial capacity (16) and load factor (0.75).
     */
    public HashSet() { map = new HashMap<>(); }
```

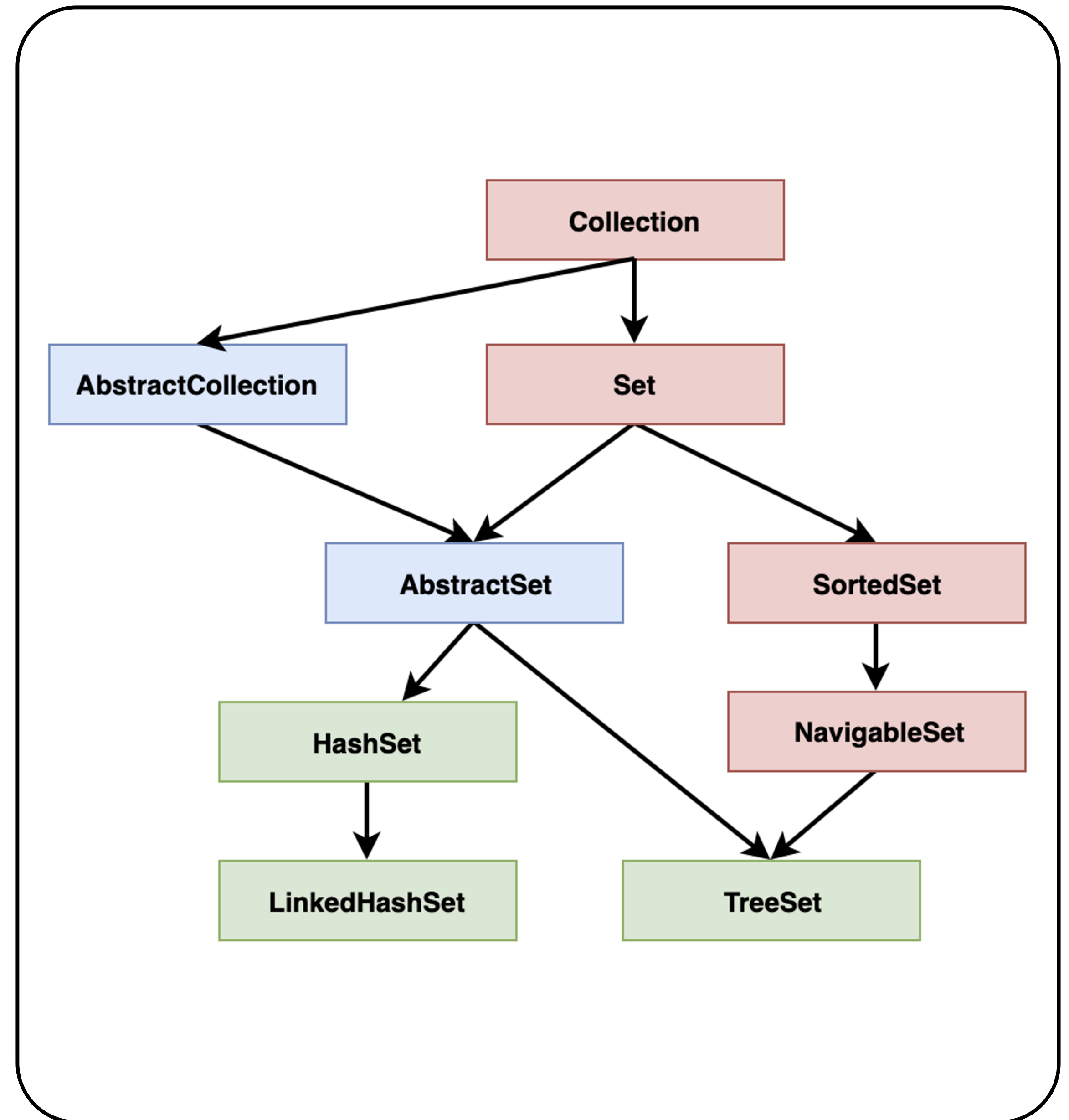
Представление HashSet

(Dummy – объект-заглушка)



Интерфейс Set

HashSet — это класс, коллекция неповторяющихся элементов, реализующая интерфейс **Set**



Ключевые методы интерфейса Set

- **add(e)** — добавляет элемент в множество, если такого ещё нет. Возвращает `true` при успешном добавлении
- **addAll(collection)** — добавляет все элементы из коллекции в множество, если такого ещё нет. Возвращает `true` при успешном добавлении
- **clear()** — удаляет все элементы из множества
- **contains(o)** — возвращает `true`, если переданный объект есть в множестве
- **containsAll(collection)** — возвращает `true`, если все объекты из переданной коллекции есть в множестве
- **isEmpty()** — возвращает `true`, если сет пустой

Ключевые методы интерфейса Set

- **iterator()** — возвращает итератор по элементам сета
- **remove(e)** — удаляет элемент из множества. Вернёт `true`, если такой элемент был
- **removeAll(collection)** — удаляет из множества все элементы, которые есть в коллекции. Вернёт `true`, если сет был изменён в результате вызова метода
- **retainAll(collection)** — оставляет в сете только элементы, которые есть в коллекции, удаляет все остальные. Вернёт `true`, если сет был изменён в результате вызова метода
- **size()** — возвращает количество элементов в множестве
- **toArray()** — возвращает массив, содержащий все элементы множества

Пример использования HashSet

Пример использования класса **String** в качестве типа объектов хранения:

```
public static void main(String[] args) {  
    Set<String> set = new HashSet<>();  
    set.add("Petya");  
    set.add("Olya");  
    set.add("Petya");  
    System.out.println(set);  
    System.out.println(set.contains("Olya"));  
}  
}
```

Сравнение HashSet с ArrayList

	HashSet	ArrayList
Сохранение порядка элементов	Нет	Да
Возможность хранить дубликаты	Нет	Да
Скорость поиска	$O(1)$	$O(n)$